



TARTU ÜLIKOOL



Code Comprehension Through Structural Encoding

Author: Philip Paškov

Supervisors:
Kallol Roy, PhD;
Zofia Mizgalewicz, MSc

Contents

1. The Problem
2. Methodology
3. Research Questions
4. Model Designs
5. Results
6. Key Findings

Motivations

- Standard tokenization treats code as linear sequence of tokens:
 - split by frequency;
 - structure is lost.
- Bottlenecks in LLMs for Code Comprehension.
- MATLAB is underrepresented language.

```
function result = add(a, b)  
    result = a + b;  
end
```

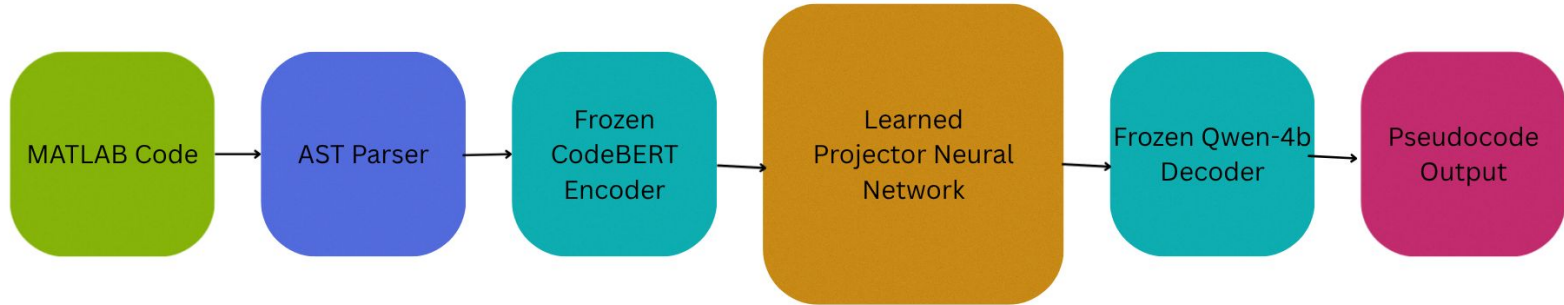
gpt-4o Tokens: ["function", "_result", "_=", "_add", "(a", ",", "_b", ")", "___", "_result", "_=", "_a", "_+", "_b", ";", "end"]

Research Questions

- **RQ1:** Does natural language processing like tokenization work for code?
- **RQ2:** Does code structure complement code sequence in understanding complex programs?
- **RQ3:** Is representing code as embedding patches, inspired by Vision Transformer architectures, an effective strategy for code comprehension?

Proposed Methodology

- MATLAB code comprehension as a pseudocode generation task.

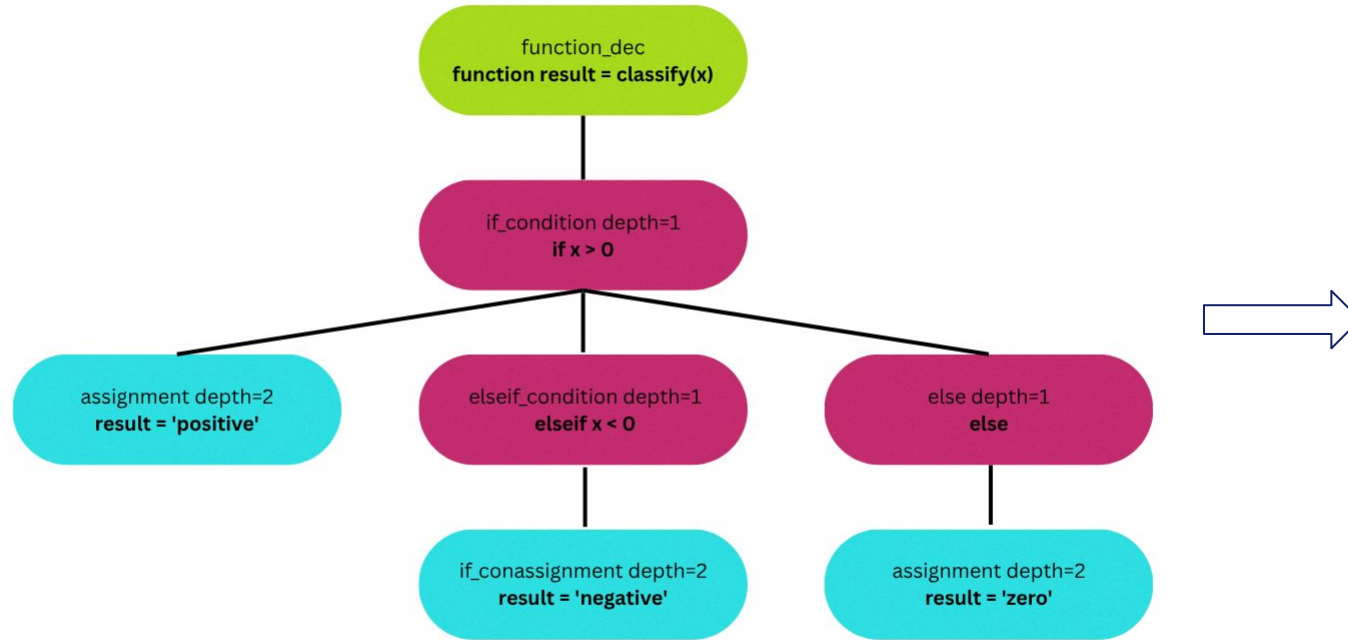


MATLAB Code

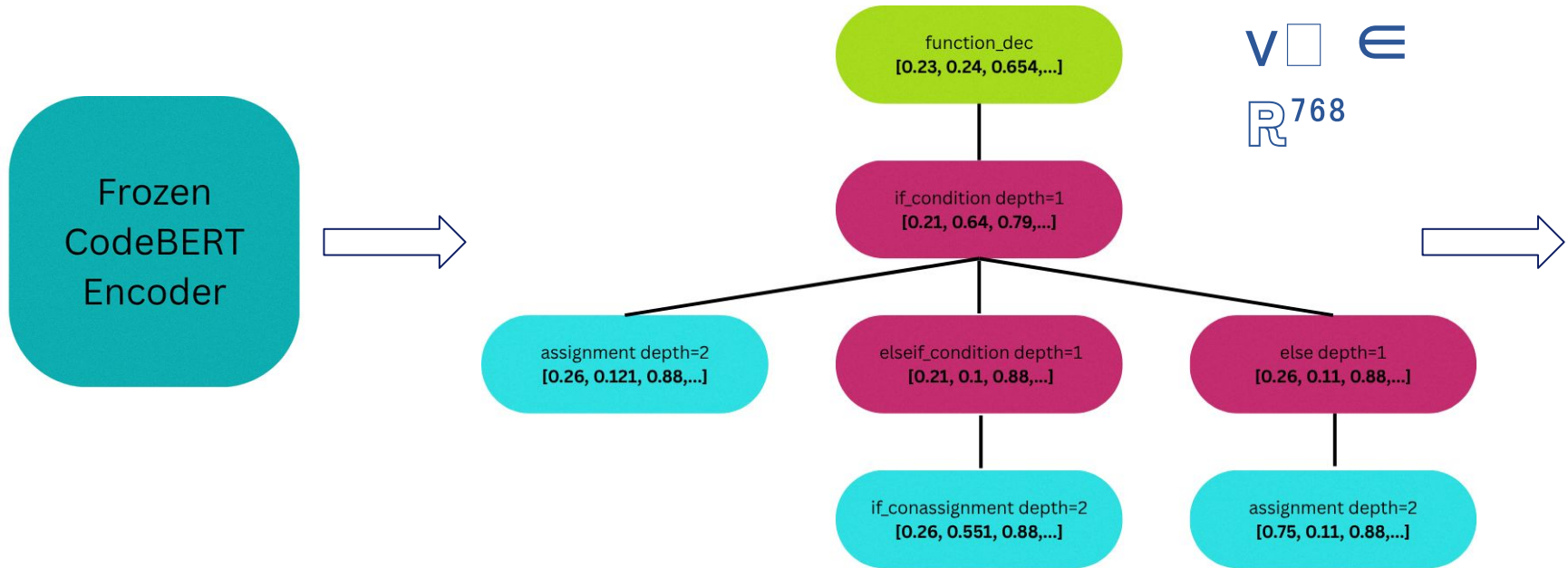
```
function result = classify(x)
    if x > 0
        result = 'positive';
    elseif x < 0
        result = 'negative';
    else
        result = 'zero';
    end
end
```



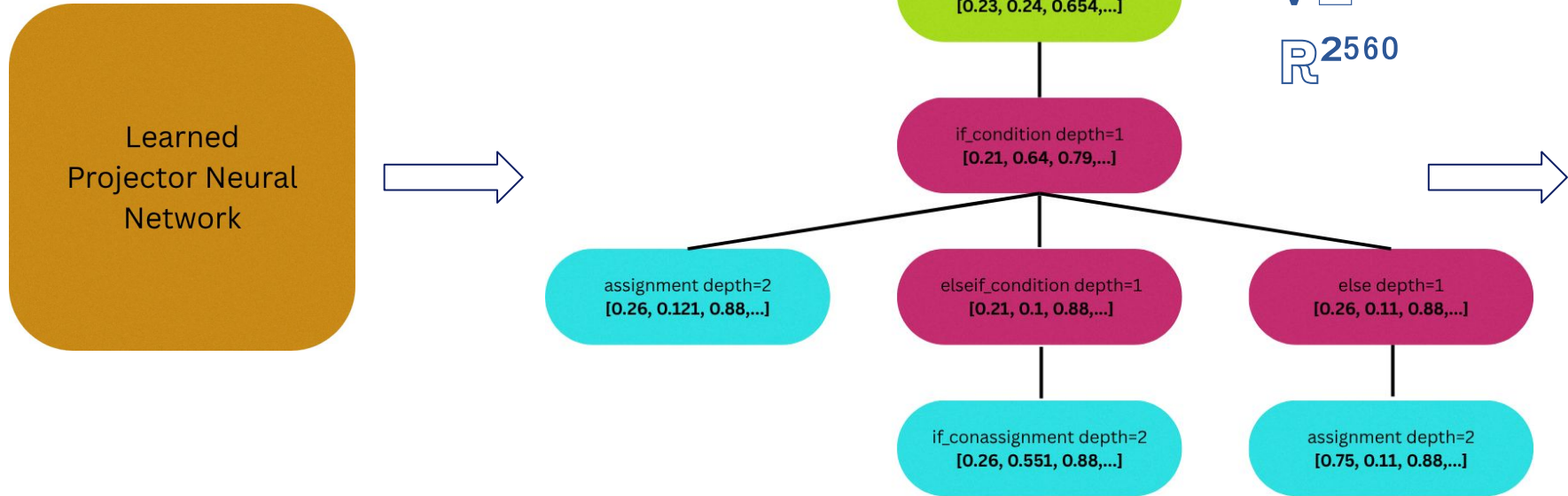
AST Parser



Frozen CodeBERT Encoder



Learned Projector



Pseudocode Output

Frozen Qwen3-
4B-Instruct
Decoder



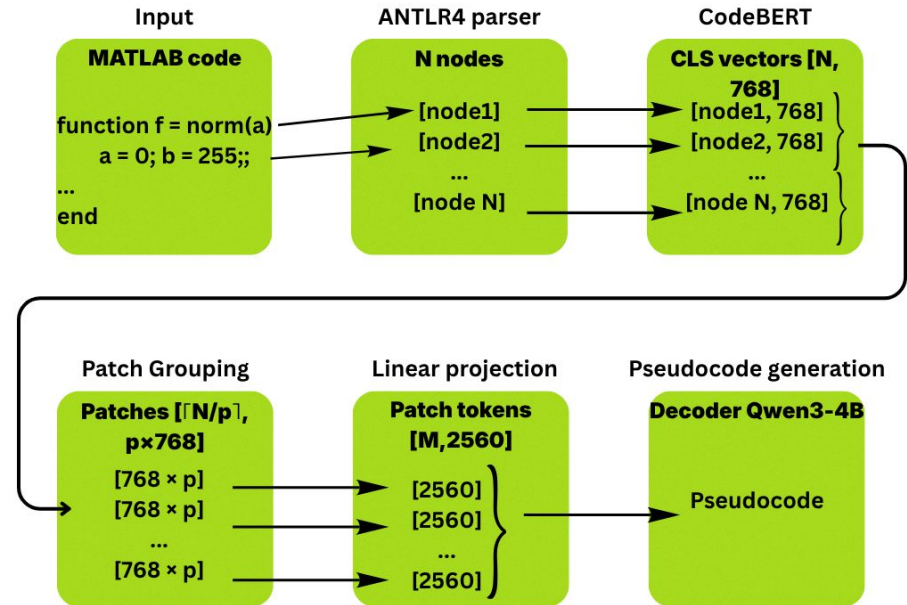
Initialize function classify with input x.
If x is greater than 0, set result to "positive".
Otherwise, if x is less than 0, set result to
"negative".
Otherwise, set result to "zero".
Return result.

Methodology

- 4 architectures developed:
 - Text-only;
 - ViT inspired patching;
 - RvNN + Text;
 - Combined (RvNN + ViT).
- LLM generated data.

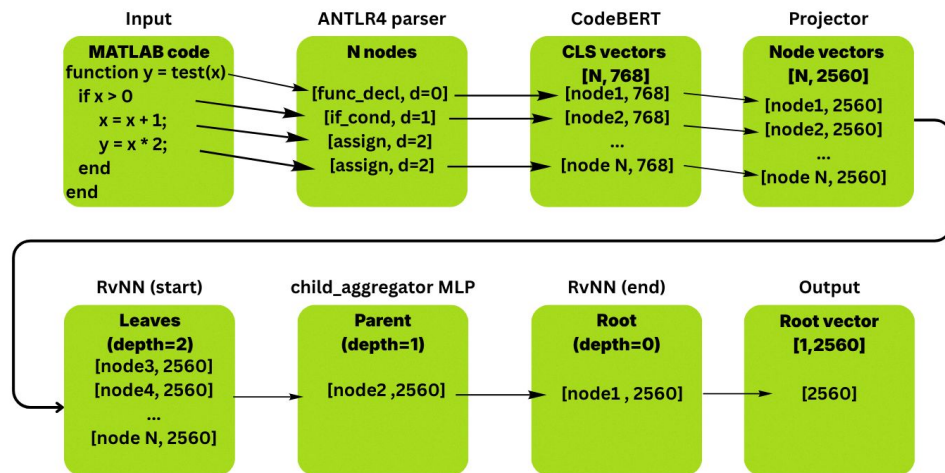
ViT

- AST parses MATLAB into N nodes.
- CodeBERT encodes each node into vector $([N, 768])$.
- Nodes are patched into groups $(M \times 768 = 1 \text{ patch})$.
- Patches are projected into 2560d vector.
- Projected patches are fed to Qwen



RvNN

- AST parses MATLAB into N nodes.
- CodeBERT encodes each node into vector ([N,768]).
- Creating tree.
- The RvNN reads the AST bottom-up, combining children into their parent one level at a time.
- Whole code is represented into 2560d vector.



Model Variants (make more attractive)

Text-only

- End-to-end Qwen native pipeline.

RvNN + Text

- Decoder sees full text + extra structural vector

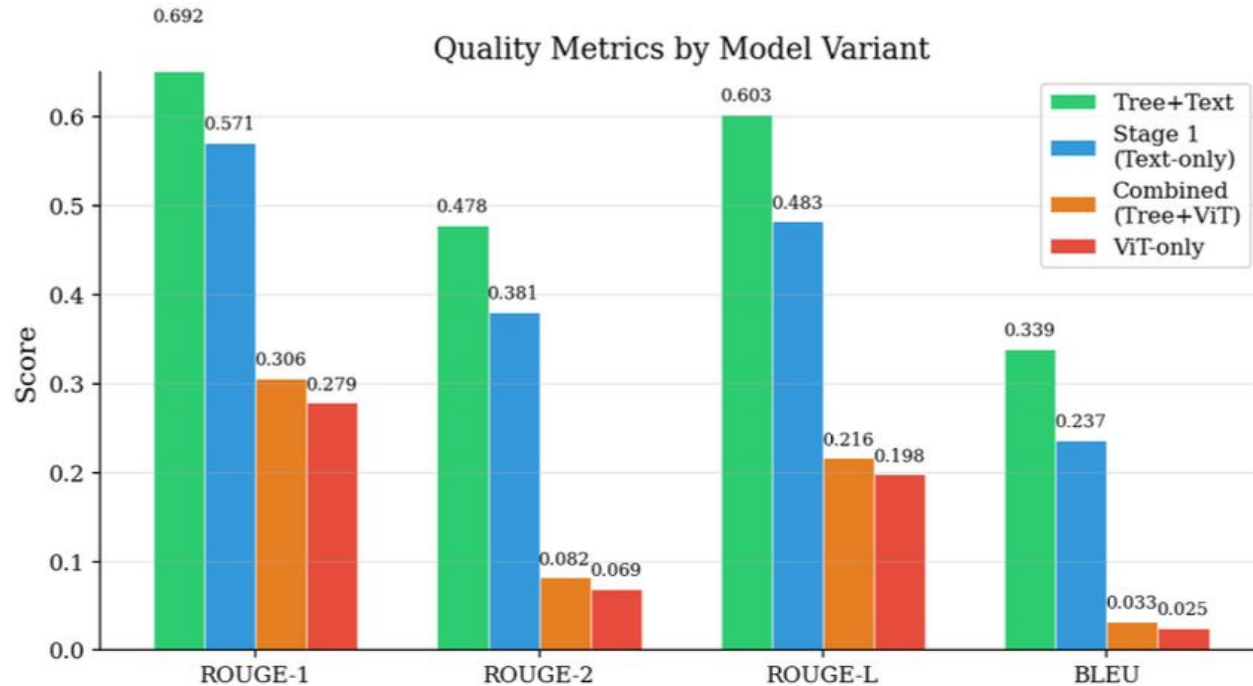
ViT patching

- Linear patching mechanism.

Combined RvNN + ViT

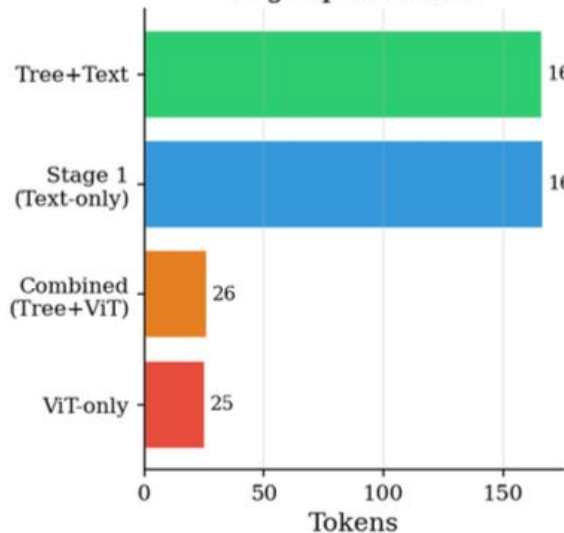
- ViT path
- RvNN path

Results

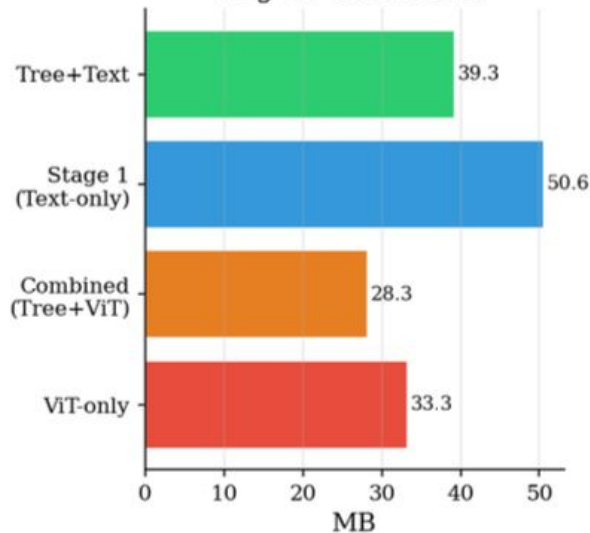


Results

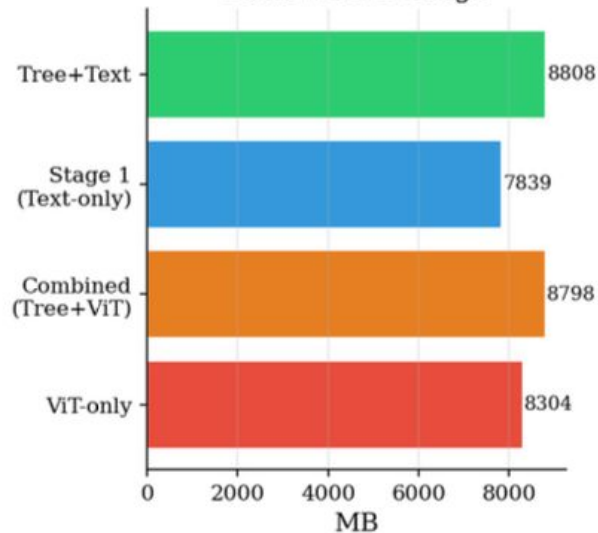
Avg Input Tokens



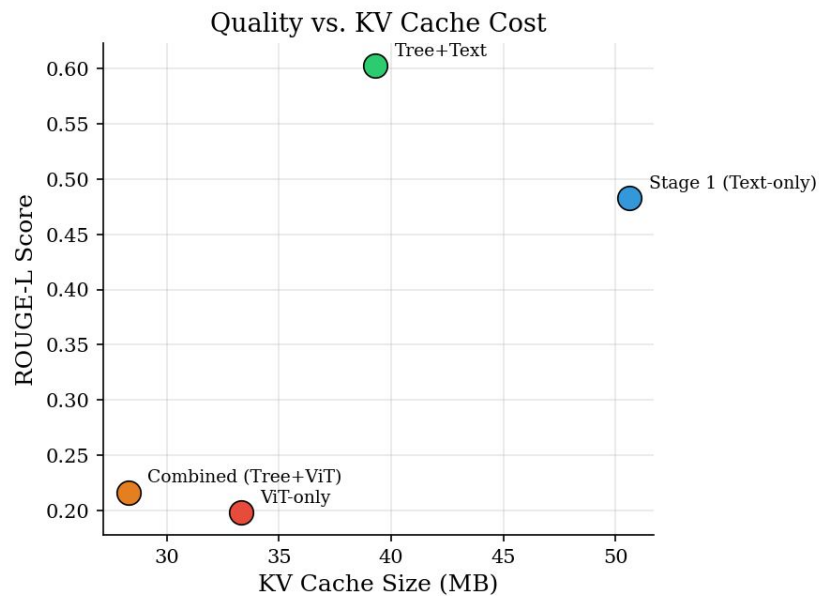
Avg KV Cache Size



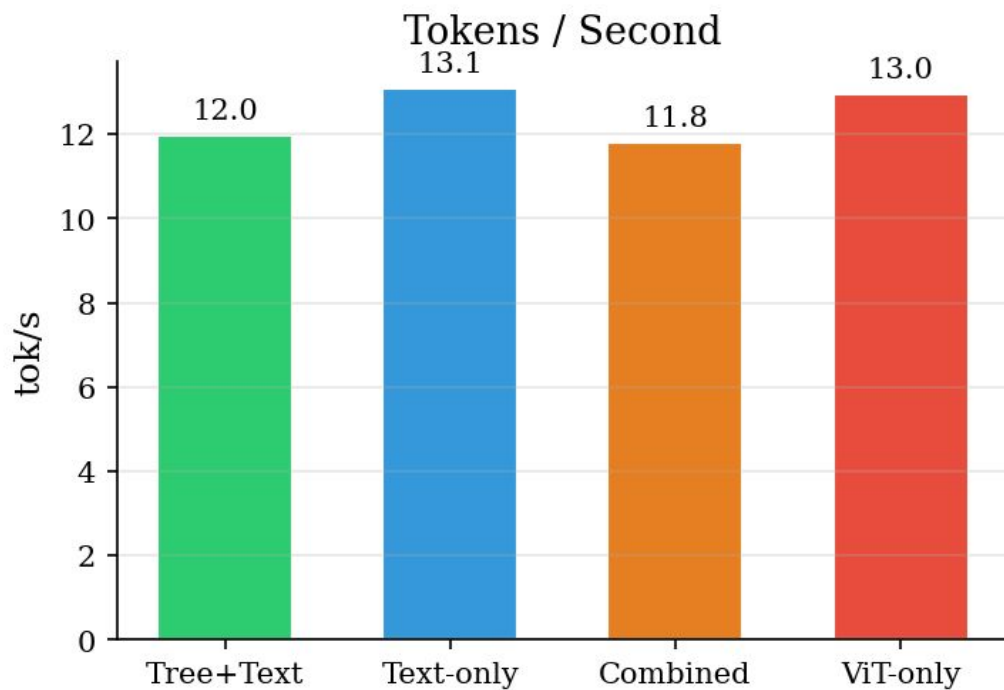
Peak VRAM Usage



Results



Results



Research Hypothesis Answering

- RQ1: Does tokenization work for code?

Reasonably well, ROUGE-1 0.571

- RQ2: Does code structure complement code sequence?

Yes, consistently improves performance across all encoding strategies.

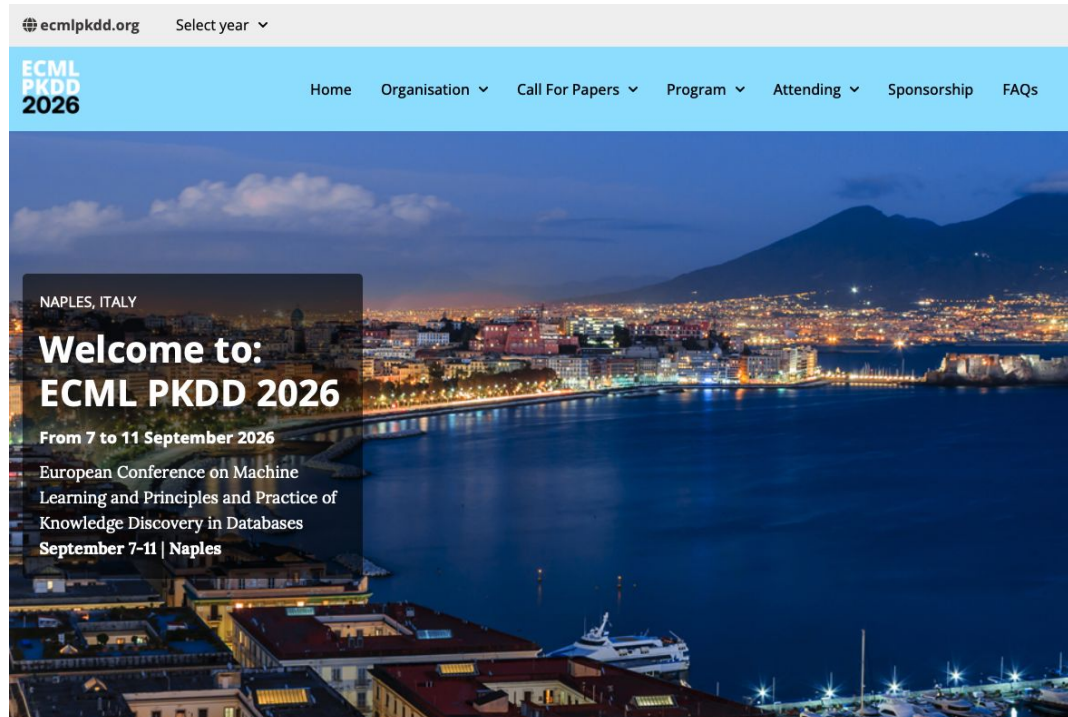
- RQ3: Is ViT patch encoding optimal for code?

No, patch-based encoding disrupts information

Use of AI in Different Stages of the Thesis

Stage of work	Use of AI
Developing the idea and planning one's work	L1- Minor assistance (generating ideas, explaining concepts).
Literature review and explaining the background	L1- Minor assistance with finding potential articles.
Methodology and solution	L1 - Assistance with formulating the core methodology.
Software development	L2 - Coding assistance in developing the solution (Gemini, Claude, Cursor).
Analysis and conclusions	L0
Text and formatting	L1 - Polishing and rephrasing in more academic manner.

ECML 2026 PKDD





Thank you for listening!

Output

Model	ROUGE-L	Output
Reference	—	Assign zero to variable <i>a</i> . Assign two hundred fifty-five to variable <i>b</i> . Find the minimum value across all elements of the input image and assign it to the variable <i>c</i> . Find the maximum value across all elements of the input image and assign it to the variable <i>d</i> .
Tree+Text	0.764	Assign zero to variable <i>a</i> . Assign 255 to variable <i>b</i> . Find the minimum value in all elements of <code>img</code> and assign it to <i>c</i> . Find the maximum value in all elements of <code>img</code> and assign it to <i>d</i> .
Text-only	0.702	Define a function named <code>normalize</code> that accepts an input <code>img</code> . Assign zero to variable <i>a</i> . Assign two hundred fifty-five to variable <i>b</i> . Find the minimum value across all elements of <code>img</code> and assign it to the variable <i>c</i> .
Combined	0.318	Assign the first element of input <i>x</i> to variable x_0 . Assign the second element of input <i>x</i> to variable x_1 . Calculate the third element of <i>x</i> by multiplying x_0 by x_1 and adding 1.
ViT-only	0.233	Determine the number of rows and columns of matrix <i>A</i> . Initialize vector <i>x</i> with zeros, having the same number of rows as <i>A</i> . Begin a loop that iterates from one to the number of columns. Inside the loop, prompt the user to input a value.